

Kick-off and first contact

UE ISD-1020 · Machine Learning / Deep Learning

Stéphane Rivaud

Université Paris-Saclay · Master ISD (apprenticeship)

14 September 2026

Who is in the room

- **UE ISD-1020** “Machine Learning / Deep Learning”
- Master Informatique, **ISD** track, apprenticeship
- Université Paris-Saclay · **PUIO B210**
- Teaching team: Stéphane Rivaud, Solal Nathan and Dylan Séchet
- Today: **13:30–17:00**

Telecom

Automotive

Power grid

Public transport

The four datasets of this module
are **your jobs**.

What this module does

The promise

You take **decisions** from industrial data. A model has value only if it **beats a baseline** and we know **where it goes wrong**. Bridge scikit-learn → neural networks, then time series. Deliverable: an **8-minute** oral exam that you can defend.

Telecom: who will churn

Definition

We must decide, for each customer, whether to follow up. The target is $\text{Churn} \in \{\text{Yes}, \text{No}\}$. The costly error is a missed Yes (a false negative). We therefore score **recall of Yes**, not accuracy.

The table

Telco Customer Churn (IBM)
7,043 customers · 26.5% Yes

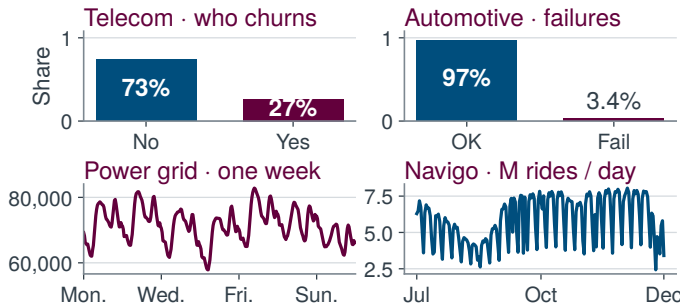
You touch it

Lab from 15:00, **today**
Always No: acc. 0.735 · recall **0**

Four jobs, one same protocol

Definition

The same six-step protocol is applied to four jobs. Classification jobs are scored with recall of the rare class; forecast jobs are scored with MAE on a time split. A model is reported only against a baseline computed on the same split.



The data

Definition

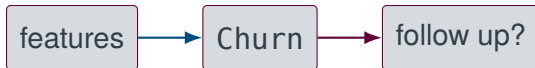
An **example** is one row of a table. Features are the columns we would know at decision time; the target is the column we want to predict. Before any model we look at types, missing values, and the class mix. This table: 7,043 rows, 21 columns, 11 missing TotalCharges (all with tenure = 0).



The target

Definition

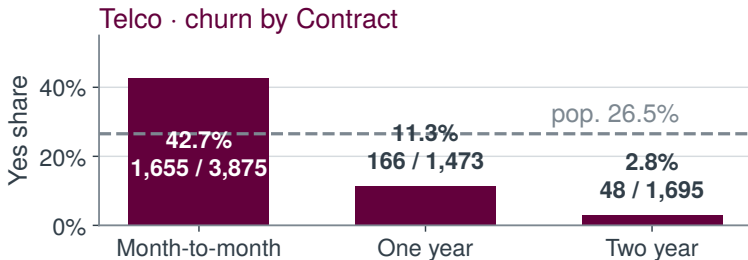
The **target** y is the column we commit to predict. It is not “another feature”: it names the decision. Here $y = \text{Churn} \in \{\text{Yes}, \text{No}\}$ and $P(y = \text{Yes}) = 1,869/7,043 = 26.5\%$. Every later metric is a statement about y on a held-out set.



The features

Definition

A **feature** is a column known at decision time that is neither the target nor a rewrite of the target. tenure, Contract, MonthlyCharges vary with Churn. customerID is an identifier (7,043 distinct values on 7,043 rows) and cannot generalize. A column that *is* the target in disguise is **leakage**: the test score will not survive production.



A supervised prediction problem

Definition

A **supervised** prediction problem is a table of examples (x_i, y_i) in which y is known at training. We fit a rule f on a training set and judge $f(x)$ only on examples that were not used to fit f (nor to prepare x). The output of f is used to take a decision.

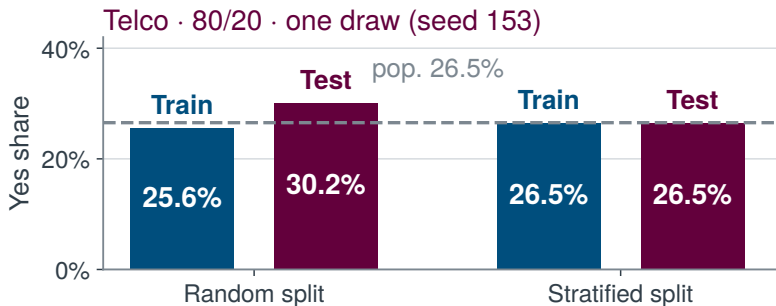
This module

Almost every job here is of this kind. D2 anomalies are the exception: sometimes there is no clear label.

Train, test, and stratification

Definition

Report the metric only on **held-out** rows. A **stratified** split locks the class mix (26.5% on every seed; unstratified 24.3–29.3% across 40 seeds).



The metric

Definition

Accuracy = $(TN + TP)/n$. **Recall of Yes** = $TP/(TP + FN)$. A missed Yes is the costly error: choose recall **before** the model.

Always No · acc. 0.735 · rec. 0.000

True	stays	1,035	0
	churns	374 FN	0
		stays churns	
		Predicted	

The metric

Definition

Accuracy = $(TN + TP)/n$. **Recall of Yes** = $TP/(TP + FN)$. A missed Yes is the costly error: choose recall **before** the model.

Always No · acc. 0.735 · rec. 0.000

True	stays	1,035	0
	churns	374 FN	0
		stays	churns
		Predicted	

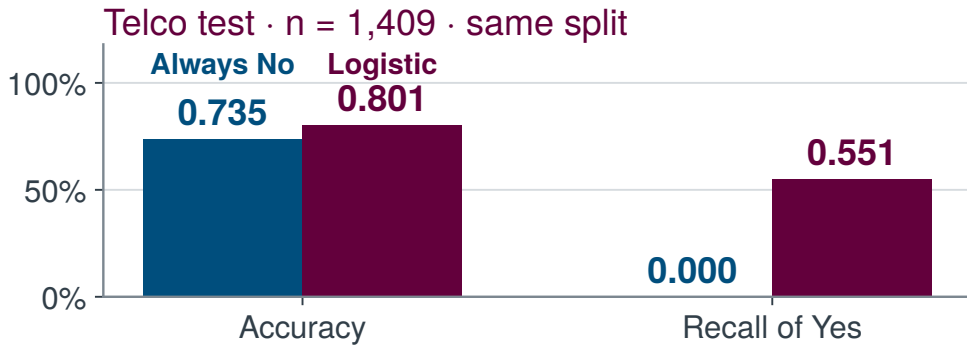
Logistic · acc. 0.801 · rec. 0.551

True	stays	923	112
	churns	168 FN	206
		stays	churns
		Predicted	

The metric

Definition

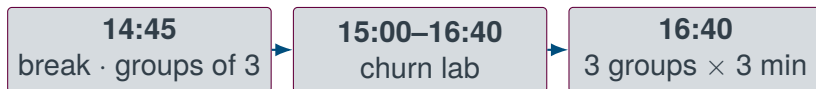
A **baseline** uses no learned model, same metric, same split. Always No ($n = 1,409$): accuracy 0.735, recall of Yes = 0.



The four Mondays

Teaching team: Stéphane Rivaud, Solal Nathan and Dylan Séchet.

D0 · 14 Sep.	D1 · 21 Sep.	D2 · 28 Sep.	D3 · 5 Oct.
13:30–17:00 First contact (churn)	09:00–12:30 13:30–17:00 Tables, then MLP	full day Forecasting, anomalies	full day Practical AM orals PM



Mix of companies encouraged · you may still **change group** until **28 September**; the group you are in that day is the one assessed on 5 October.

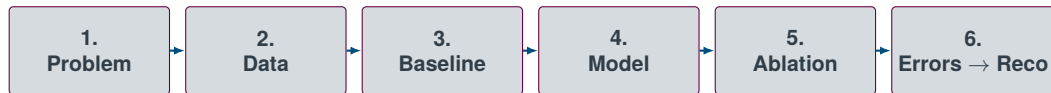
What this module does not cover

Not this UE

- **NLP**, LLM **fine-tuning**
- **Reinforcement** learning
- **Generative** models
- **Convex optimisation** theory
- Ethics / fairness: one “limits” slide, not a course

If your job is NLP, that is not this UE. No GPU promise.

The six steps



1. **Problem** — decision, target, metric, split
2. **Data** — types, leakages, quality
3. **Baseline** — naive floor, quantified
4. **Model** — one justified family, a score on the hold-out
5. **Ablation** — what actually changes the score?
6. **Error analysis** → **Recommendation** — where it goes wrong, what do we do?

The six oral slides use these six headings. Every notebook ends with these six cells.

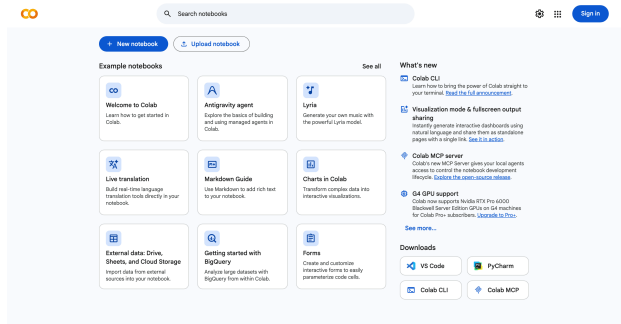
The protocol on churn

The six steps, instantiated

1. **Problem** — who will churn? Metric: **recall** of Yes
2. **Data** — 7,043 customers, $\sim 26\%$ Yes; no customerID
3. **Baseline** — Always No: accuracy **73%**, recall **0**
4. **Model** — logistic regression, **this afternoon**
5. **Ablation** — without Contract, does the score change?
6. **Error analysis** → **Recommendation** — where are the FN? Do we deploy?

Google Colab

- Browser + **Google account** · no local install
- **CPU** is enough · no GPU required
- Link given at the start of the session
- Then File → Save a copy in Drive
- Run the **setup cell**: it puts common/ and data/ in place



<https://colab.research.google.com/>



Pipeline

```
class sklearn.pipeline.Pipeline(steps, *, transform_input=None, memory=None,
verbose=False)
```

[\[source\]](#)

A sequence of data transformers with an optional final predictor.

`Pipeline` allows you to sequentially apply a list of transformers to preprocess the data and, if desired, conclude the sequence with a final [predictor](#) for predictive modelling.

Intermediate steps of the pipeline must be transformers, that is, they must implement `fit` and `transform` methods. The final [estimator](#) only needs to implement `fit`. The transformers in the pipeline can be cached using `memory` argument.

The purpose of the pipeline is to assemble several steps that can be cross-validated together while setting different parameters. For this, it enables setting parameters of the various steps using their names and the parameter name separated by a `'_'`, as in the example below. A step's estimator may be replaced entirely by setting the parameter with its name to another estimator, or a transformer removed by setting it to `'passthrough'` or `None`.

For an example use case of `Pipeline` combined with [GridSearchCV](#), refer to [Selecting dimensionality reduction with Pipeline and GridSearchCV](#). The example [Pipelining: chaining a PCA and a logistic regression](#) shows how to grid search on a pipeline using `'_'` as a separator in the parameter names.

Read more in the [User Guide](#).

```
sklearn.pipeline.Pipeline from sklearn.pipeline import Pipeline
```

PyTorch: nn.Module

 **v2.14...** [Install PyTorch](#) [User Guide](#) [Reference API](#) [Developer Notes](#) [Community](#) [Tutorials](#) [Go to pytorch.org](#)

Search the docs ... 

[Home](#) > [Reference API](#) > [torch.nn](#) > [Module](#) [Rate this Page](#) 

Module

`class torch.nn.Module(*args, **kwargs)` [\[source\]](#)

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

`torch.nn.Module` `loss.backward(); optimizer.step()`

What works with an LLM

Allowed **everywhere**: lectures, labs, mini-project, oral exam.

Ask this

Explain this sklearn / PyTorch error and propose one lead, without rewriting the notebook. Live demo: precision, recall and PR-AUC when the FN costs more.

How to ask

An **error message**, not the whole notebook. A **comparison** of metrics, not “do the lab”. “Ask me three questions as if you were the **jury**”. Reread, rerun, keep only what you can **explain**.

What misleads

Three traps

- An **invented API**, delivered with confidence
- A **hyperparameter** that does not exist
- Code that **runs** but **leaks** the target

```
scaler = StandardScaler().fit(X)    # X contains the test  
leaked = scaler.transform(X)  
X_train, X_test, y_train, y_test = train_test_split(leaked, y)
```

The module rule. Nothing to declare: no form, no cell, no list of prompts. Whatever you hand in, **you wrote it** — and you can explain every line. What costs points: a line you **cannot explain**. Not the tool.

The oral exam of 5 October

The day

Mini-project in the **morning**
(5 Oct., 09:00–12:30)
oral exam in the **afternoon**

On the clock

8 min of talk + **4 min** of questions
(12 min on the clock)

The jury

Groups of **3** · up to **14** groups
one jury

No separate exam

Entire continuous assessment:
everything happens **inside the slots**

What is assessed

The oral exam

The **protocol** you followed, the method, what you found in your errors, and your answers to the jury

What you hand in

A notebook that **runs** end to end, and **six slides** — the six steps, same headings

A notebook that does not run is not saved by the oral exam.

The exact weighting is being settled by the teaching team and will be given to you before 28 September.

Open the notebook

1. Shared course folder / link shared at the start of the session (not a GitHub badge)
2. File → Save a copy in Drive
3. Two variants: default **autonomous** (guided if the room needs hints)
4. Run the **setup cell** at the top before anything else

Attributions and licences

Module material: Stéphane Rivaud.

Teaching team: Stéphane Rivaud, Solal Nathan and Dylan Séchet.

Tutorials

- scikit-learn MOOC · CC BY 4.0
- PyTorch tutorials · BSD-3-Clause
- d2l.ai · CC BY-SA 4.0
- micrograd · MIT

Datasets

- Telco IBM · Apache-2.0
- AI4I UCI · CC BY 4.0
- éCO2mix · Licence Ouverte v2.0
- IDFM · ODbL · NAB · MIT

Documentation

- scikit-learn · BSD-3-Clause
- PyTorch · BSD-3-Clause
- Google Colab
- Keras · Apache-2.0

PyTorch, the PyTorch logo and related marks are trademarks of The Linux Foundation. scikit-learn, Colab and Keras marks remain with their owners; used here only to identify the tools.